



MKD

2026-06-26

- 1 Contest
- 2 Data structures
- 3 Number theory
- 4 Geometry
- 5 Strings
- 6 Various

Contest (1)

dbg.h20 lines

```
#define dbg(...) cerr<<"DEBUG:["<<#__VA_ARGS__<<"] = [" ,\
    _print(__VA_ARGS__),cerr<<"]\n"
template<class T, class=void>struct cntr : false_type {};
template<class T>struct cntr<T,
    void_t<decltype(declval<T>().begin()),
    decltype(declval<T>().end()),
    typename T::value_type> : true_type {};
template<class T>using raw=remove_cv_t<remove_reference_t<T> >;
template<class T>enable_if_t<!cntr<T>::value>
    _print(const T &x) { cerr << x; }
template<class T>enable_if_t<cntr<T>::value>_print(const T &v){
    cerr << "["; bool f = false;
    for (const auto &x: v)
        f ? cerr << (cntr<raw<decltype(x)> >::value ? ",\n":", "):
            cerr, f = true, _print(x);
    cerr << "]" ;
}
template<class T, class... A>
void _print(const T &x, const A &... a) {
    _print(x);(cerr << " , ", _print(a)), ...;}
```

template.cpp25 lines

```
#include <bits/stdc++.h>
using namespace std;
// #define LOCAL_DEBUG
#ifdef LOCAL_DEBUG
#include "dbg.h"
#else
#define dbg(...) 42
#endif
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
void init() {}
void solve() {}
signed main() {
    init();
    cin.tie(nullptr), ios::sync_with_stdio(false);
    // cin.exceptions(cin.failbit);
    int T = 1;
    // cin >> T;
    while (T--)solve();
    return 0;
}
```

1.1 checker

cek.sh19 lines

```
#!/usr/bin/env bash
set -u
cd "$(dirname "$0")"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
cnt=0
while true; do
    cnt=$((cnt + 1))
    echo "Testing case #$cnt"
    ./gen "${1-}" > in.txt
    ./std < in.txt > out1.txt
    ./sol < in.txt > out2.txt
    if ! diff -q out1.txt out2.txt > /dev/null; then
        echo "WA on case #$cnt"
        diff out1.txt out2.txt
        read -r -p "Press Enter to continue..."
    fi
done
```

cek.bat19 lines

```
@echo off
cd "%~dp0"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
set cnt=0
:loop
set /a cnt+=1
echo Testing case %cnt%
gen %1 > in.txt
std < in.txt > out1.txt
sol < in.txt > out2.txt
fc out1.txt out2.txt > nul
if errorlevel 1 (
    echo WA on case %cnt%
    fc out1.txt out2.txt
    pause
)
goto loop
```

gen.h22 lines

```
uint64_t seed = chrono::steady_clock::now()
    .time_since_epoch().count() ^ random_device {} ();
mt19937 rng(seed);
template<class T>
T randi(T l, T r) {return uniform_int_distribution<T>(l, r)(rng); }
vi randvi(int n, int l, int r) {
    vi v(n);
    rep(i, 0, sz(v))v[i] = randi(l, r);
    return v;
}
vi randp(int n) {
    vi p(n);
    iota(all(p), 1);
    shuffle(all(p), rng);
    return p;
}
int main(int argc, char *argv[]) {
    if (argc >= 2)
        seed = stoull(argv[1]), rng.seed(seed);
    cerr << seed << endl;
    return 0;
}
```

Data structures (2)

UnionFind.h

Description: 并查集数据结构。
Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```
struct UF {
    vi e;
    UF(int n) : e(n, -1) {}
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        e[a] += e[b]; e[b] = a;
        return true;
    }
};
```

RMQ.h

Description: 区间最小值查询。返回 $\min(V[a], V[a + 1], \dots V[b - 1])$ 。
Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V| \log |V| + Q)$

510c32, 15 lines

```
template<class T>struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) { // pw = 2^{k-1}
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, 0, sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

FenwickTree.h

Description: 树状数组。计算前缀和 $a[0] + a[1] + \dots + a[pos - 1]$ ，并通过计算新旧值之差更新单个元素 $a[i]$ 。
Time: Both operations are $\mathcal{O}(\log N)$.

219f0b, 21 lines

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >>= 1)
            if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
                pos += pw, sum -= s[pos-1];
        return pos;
    }
};
```

SubMatrix.h

Description: 在给定左上角和右下角的情况下，快速计算子矩阵和。坐标采用左闭右开区间。

Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // 左上角四个区间

Time: $O(N^2 + Q)$

c59ada, 13 lines

```
template<class T>
struct SubMatrix {
    vector<vector<T>>> p;
    SubMatrix(vector<vector<T>>& v) {
        int R = sz(v), C = sz(v[0]);
        p.assign(R+1, vector<T>(C+1));
        rep(r,0,R) rep(c,0,C)
            p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
    }
    T sum(int u, int l, int d, int r) {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};
```

Number theory (3)

3.1 整除性

欧几里得

$$\gcd(a, b) = 1$$

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

找符合 $ax + by = d = \gcd(a, b)$ 的一组 x, y

$$a \cdot a_{\text{inv}} + py = 1 = \gcd(a, p)$$

$$by + (a \bmod b)x = d$$

$$ax + b(y - \left\lfloor \frac{a}{b} \right\rfloor x) = d$$

euclid.h

Description: 找到两个整数 x 和 y , 使得 $ax + by = \gcd(a, b)$ 。如果你只需要 gcd, 可以使用内置的 `__gcd`。如果 a 和 b 互质, 那么 x 就是 $a \bmod b$ 的逆元。
Time: $O(\log \min(a, b))$

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

中国剩余定理

定理 CRT. 设 $m_1, m_2, \dots, m_n$ 两两互质, 且

$$(S): \begin{cases} x \equiv a_1 \pmod{m_1}, \\ x \equiv a_2 \pmod{m_2}, \\ \vdots \\ x \equiv a_n \pmod{m_n}. \end{cases}$$

令

$$M = \prod_{i=1}^n m_i, \quad M_i = \frac{M}{m_i}, \quad t_i \equiv M_i^{-1} \pmod{m_i}.$$

则方程组 (S) 有解, 并且一组解为

$$x \equiv \sum_{i=1}^n a_i t_i M_i \pmod{M}.$$

构造性证明. 由于 $m_1, m_2, \dots, m_n$ 两两互质, 对任意 i , 有

$$\gcd(M_i, m_i) = 1.$$

因此存在 t_i , 使得

$$t_i M_i \equiv 1 \pmod{m_i}.$$

令

$$x = \sum_{k=1}^n a_k t_k M_k.$$

固定任意 i . 当 $k \neq i$ 时, 由于 $m_i \mid M_k$, 有

$$a_k t_k M_k \equiv 0 \pmod{m_i}.$$

于是

$$\begin{aligned} x &= a_i t_i M_i + \sum_{k \neq i} a_k t_k M_k \\ &\equiv a_i \cdot 1 + \sum_{k \neq i} 0 \pmod{m_i} \\ &\equiv a_i \pmod{m_i}. \end{aligned}$$

因此 x 同时满足所有同余式, 故 x 是方程组 (S) 的一组解。

CRT.h

Description: 中国剩余定理。

`crt(a, m, b, n)` 计算 x , 使得 $x \equiv a \pmod{m}, x \equiv b \pmod{n}$ 。如果 $|a| < m$ 且 $|b| < n$, 则 x 将满足 $0 \leq x < \text{lcm}(m, n)$ 。假设 $mn < 2^{62}$ 。

Time: $\log(n)$

"euclid.h"

04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

3.2 模运算

ModularArithmetic.h

Description: 用于模运算, 设置 MOD 后使用。

"euclid.h"

2f9ed0, 18 lines

```
constexpr ll MOD = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll _x) : x((_x % MOD + MOD) % MOD) {}
    Mod operator+(Mod b) { return Mod(((x + b.x)%MOD+MOD)%MOD); }
    Mod operator-(Mod b) { return Mod(((x - b.x)%MOD+MOD)%MOD); }
    Mod operator*(Mod b) { return Mod((x * b.x %MOD+MOD)%MOD); }
    Mod operator/(Mod b) { return *this * invert(b); }
    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, MOD, x, y);
        assert(g == 1); return Mod((x % MOD + MOD) % MOD);
    }
    Mod operator^(ll e) {
        if (!e) return Mod(1);
        Mod r = *this ^ (e / 2); r = r * r;
        return e & 1 ? *this * r : r;
    }
};
```

模逆元

$$\frac{a}{b} \equiv a \cdot b_{\text{inv}} \equiv x \pmod{p}$$

$$\Rightarrow a \equiv bx \pmod{p}$$

$$\Rightarrow a \cdot b_{\text{inv}} \cdot b \equiv bx \pmod{p}$$

$$\Rightarrow a \cdot (b_{\text{inv}} \cdot b - 1) \equiv 0 \pmod{p}$$

$$\Rightarrow b_{\text{inv}} \cdot b \equiv 1 \pmod{p}$$

$$\text{inv存在} \Leftrightarrow \gcd(b, p) = 1$$

$$b \cdot b_{\text{inv}} = k \cdot p + 1, \quad g = \gcd(b, p) \Rightarrow g \cdot b_1 \cdot b_{\text{inv}} = k \cdot g \cdot p_1 + 1$$

$$\Rightarrow g \cdot (b_1 \cdot b_{\text{inv}} - k \cdot p_1) = 1$$

$$\text{数域为 } \mathbb{Z} \Rightarrow g = 1, \quad b_1 \cdot b_{\text{inv}} - k \cdot p_1 = 1$$

ModInverse.h

Description: 预计算模逆元, 假设 $\text{LIM} \leq \text{mod}$ 且 mod 是质数。

17b468, 4 lines

```
constexpr ll MOD = 1000000007, LIM = 200000;
vector<int> inv(LIM + 1, 0); inv[1] = 1;
for (int i = 2; i <= LIM; i++)
    inv[i] = MOD - (MOD / i) * inv[MOD % i] % MOD;
```

Geometry (4)

4.1 Geometric primitives

CW ::= ClockWise 顺时针

CCW ::= CounterClockWise 逆时针

Point.h

Description: 平面点类。T 为 double 或 long long 等类型 (避免 int)。

de820b, 30 lines

```
constexpr double eps = 0;
template<class T> int sgn(T x) { return (x > eps)-(x < -eps); }
template<class T> struct Point {
    using P = Point;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    bool approxEq(P p) const { return !(sgn(x-p.x)|sgn(y-p.y)); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; } // 点乘
    T cross(P p) const { return x * p.y - y * p.x; } // 叉乘
    // Oa与Ob叉乘
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x * x + y * y; } // 到原点距离平方
    double dist()const { return sqrt((double)dist2()); } // 到原点距离
    // 与x轴夹角, 范围[-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // 单位化
    P perp() const { return P(-y, x); } // 绕原点逆时针旋转九十度
    P normal() const { return perp().unit(); } // 法向量
    // 绕原点逆时针旋转a弧度
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a));
    }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
};
```

$$dis = \frac{|(e-s) \times (p-s)|}{|e-s|}$$
$$dis \leq eps \Leftrightarrow |(e-s) \times (p-s)| \leq |e-s| \cdot eps$$

sideOf.h
Description: 返回点 p 相对于线段 se 的位置。 $1/0/-1 \Leftrightarrow$ 左侧/在线上/右侧。如果给定了可选参数 eps , 则当点 p 距离线段 se 的距离小于 eps 时, 返回 0. P 应该是 Point<T> 类型, 其中 T 可以是 double 或 long long 等类型。在中间步骤中使用了乘积运算, 因此如果使用 int 或 long long 类型, 请注意溢出。
Usage: bool left = sideOf(p1,p2,q)==1;

```
"Point.h"3af81c, 9 lines
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s); // 避免 s==e
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

lineDistance.h
Description:
返回点 p 到包含点 a 和 b 的直线的有符号距离。从 a 到 b 看, 左侧为正值, 右侧为负值。a==b 时返回 nan. P 应该是 Point<T> 或 Point3D<T> 类型, 其中 T 可以是 double 或 long long 等类型。在中间步骤中使用了乘积运算, 因此如果使用 int 或 long long 类型, 请注意溢出。使用 Point3D 将始终给出非负距离。对于 Point3D, 请在叉积的结果上调用.dist。
Point.h

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

求直线交点

$$1/0/-1 \Leftrightarrow \text{唯一交点/无交点/重合}$$
$$\begin{aligned} L_1 &: s_1 + t(e_1 - s_1) \\ L_2 &: s_2 + u(e_2 - s_2) \end{aligned}$$
$$\text{交点 } X = s_1 + t(e_1 - s_1) = s_2 + u(e_2 - s_2)$$
$$\overrightarrow{s_2 X} \parallel \overrightarrow{s_2 e_2} \Rightarrow (X - s_2) \times (e_2 - s_2) = 0$$
$$\Rightarrow (s_1 + t(e_1 - s_1) - s_2) \times (e_2 - s_2) = 0$$
$$\Rightarrow (s_1 - s_2) \times (e_2 - s_2) + t(e_1 - s_1) \times (e_2 - s_2) = 0$$
$$\Rightarrow t = \frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$\begin{aligned} X &= s_1 + \frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \cdot (e_1 - s_1) \\ &= \left[\frac{(e_1 - s_1) \times (e_2 - s_2) - (e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \right] \cdot s_1 \\ &\quad + \left[\frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \right] \cdot e_1 \\ &= \left[\frac{(e_1 - s_1) \times (e_2 - s_2) + (s_1 - s_2) \times (e_2 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \right] \cdot s_1 \\ &\quad + \left[\frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \right] \cdot e_1 \cdots \\ &= \frac{[(e_1 - s_2) \times (e_2 - s_2)] \cdot s_1 + [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1}{(e_1 - s_1) \times (e_2 - s_2)} \\ &= \frac{p \cdot s_1 + q \cdot e_1}{d} \end{aligned}$$

lineIntersection.h
Description:
如果存在唯一的交点, 返回 {1, 交点}; 如果不存在交点, 返回 {0, (0,0)}; 如果存在无数个交点, 返回 {-1, (0,0)}。如果使用 Point<ll>, 且交点坐标不全为整数, 则返回错误的结果。中间步骤使用了三个坐标的乘积, 因此如果使用 int 或 ll, 请注意可能的溢出。
Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
Point.h

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

Strings (5)

KMP.h
Description: pi[x] 计算 s 的最长前缀长度, 且该前缀以 s[x] 结尾, 且不等于 s[0...x] 本身 (abacaba -> 0010123)。可以用来找到一个字符串的所有出现位置。
Time: $\mathcal{O}(n)$

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h
Description: 维护 [l, r] 表示当前已知最靠右的匹配区间 $s[l..r-1] == s[0..l-1]$ 。z[i] 计算 $s[i:]$ 与 s 的最长公共前缀, 但 $z[0] = 0$ 除外。(abacaba -> 0010301)
Time: $\mathcal{O}(n)$

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i,1,sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

Manacher.h
Description: 对于字符串中的每个位置 i, $p[0][i]$ = 中心在 i-1 与 i 之间的最长偶回文半长, $p[1][i]$ = 中心在 i 的最长奇回文半长 (向下取整)(不包括 i)
Time: $\mathcal{O}(N)$

```
array<vi, 2> manacher(const string &s) {
    int n = sz(s);
    array<vi, 2> p = {vi(n + 1, 0), vi(n, 0)};
    rep(id, 0, 2)for (int i = 0, l = 0, r = 0; i < n; i++) {
        int t = r - i + !id;
        if (i < r) p[id][i] = min(t, p[id][l + t]);
        int L = i - p[id][i], R = i + p[id][i] - !id;
        while (L - 1 >= 0 && R + 1 <= n - 1 && s[L - 1] == s[R + 1])
            p[id][i]++, L--, R++;
        if (R > r) l = L, r = R;
    }
    return p;
}
```

MinRotation.h
Description: 找到字符串中字典序最小的循环位移
Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());
Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
    int a=0, n=sz(s); s += s;
    rep(b,0,n) rep(k,0,n) {
        if (a+k == b || s[a+k] < s[b+k]) { b += max(0, k-1); break; }
        if (s[a+k] > s[b+k]) { a = b; break; }
    }
    return a;
}
```

AC 自动机

Aho-Corasick 自动机用于 多模式串匹配

给定若干模式串:

$\left\{ \begin{array}{l} \text{he} \\ \text{she} \\ \text{his} \\ \text{hers} \end{array} \right.$

再给一个文本串:ushers

要找出每个模式串在哪些位置出现。朴素做法是每个模式串跑一次 KMP, 总复杂度约为: $\mathcal{O}(\text{模式串个数} \times \text{文本长度})$

AC 自动机的目标:

构建: $\mathcal{O}(\text{所有模式串总长} \times \text{字符集大小})$

find: $O(\text{文本长度} + \text{匹配输出量})$

findAll:最坏达 $O(\text{文本长度} \times \text{模式串数})$

直接输出链: $\text{end} \rightarrow \dots \rightarrow \text{start}$,

fail 输出链: $\text{N}[y].\text{end} \rightarrow \dots$,

拼接后: $\text{end} \rightarrow \dots \rightarrow \text{start} \rightarrow \text{N}[y].\text{end} \rightarrow \dots$.

AhoCorasick.h

Description: Aho-Corasic 自动机, 用于多模式串匹配。用 AhoCorasick ac(patterns) 初始化; 自动机的起始节点下标为 0。find(word) 对每个位置返回在该位置结束的最长模式串编号; 若不存在则返回 -1。findAll(−, word) 找出从每个位置开始的所有模式串 (若无重复模式串, 数量最多为 $N\sqrt{N}$), 按长度从短到长排列。允许重复模式串; 不允许空模式串。若要找出从每个位置开始的最长模式串, 反转所有输入即可。对大字符集, 可将每个符号拆成若干块, 并用哨兵位标记符号边界。

Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

91f62d, 73 lines

```
vi encode(vi a) {
    vi v;
    for (int i = 0; i < sz(a); v.push_back(256), i++)
        rep(i, 0, 4)v.push_back((a[i] >> (8 * i)) & 256);
    return v;
}
```

```
struct AhoCorasick {
    enum { alpha = 26, first = 'A' }; // change this!
    struct Node {
        // (nmatches is optional)
        int back = 0, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int& m = N[n].next[c - first];
            if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if (N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1) {
        rep(i, 0, sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
        N.emplace_back(0);
    }
}
```

```
queue<int> q;
for (q.push(0); !q.empty(); q.pop()) {
    int n = q.front(), prev = N[n].back;
    rep(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
            N[ed].back = y;
            (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                = N[y].end;
            N[ed].nmatches += N[y].nmatches;
            q.push(ed);
        }
    }
}
```

```
}
vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}
vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

后缀数组

定义. 将字符串的所有后缀按字典序排序, 只保存后缀起点:

$\text{sa}[i]$ = 字典序第 i 小的后缀起点.

若 $\text{s}=\text{banana}$, 排序后为

a, ana, anana, banana, na, nana,

因此 $\text{sa} = [5, 3, 1, 0, 4, 2]$ 。反数组通常记为 rk :

$\text{rk}[\text{pos}]$ = 后缀 $\text{s}[\text{pos} \dots]$ 的排名.

本板子构造结束后用 $\text{x}[i]$ 承担 $\text{rk}[i]$ 的角色。

哨兵. 在 s 后加入严格小于所有字符的哨兵。本板子使用

$\text{s.push_back}(0)$

故输入不能包含 nul 字符。若原串长为 n , 则哨兵所在的空后缀最小:

$\text{sa}[0] = n$.

所以返回数组大小为 $n + 1$, 处理原串后缀时常从 $\text{sa}[1]$ 开始看。

循环位移到后缀数组. cp-algorithms 的构造本质上先排序 **循环位移** 而非直接排序后缀。循环位移把字符串看成首尾相接, 位置 i 开始的循环串为

$\text{s}[i \dots] + \text{s}[0 \dots i-1]$.

更一般地, 当区间右端绕回开头时:

$\text{s}[i..j] = \text{s}[i..n-1] + \text{s}[0..j] \quad (i > j),$

下标按模 n 理解。

排序循环位移如何变成排序后缀? 在原串后加入最小哨兵后, 任何真实后缀都可看作一个循环位移中哨兵之前的前缀。由于哨兵严格小于所有普通字符, 两个后缀第一次分出长短时, 较短者会先遇到哨兵, 字典序也正好更小。因此

sort cyclic shifts of $(s + '0') \iff$ sort suffixes of s ,

只不过结果最前面多出哨兵位置对应的空后缀。本板子不删除它, 所以保留 $\text{sa}[0] = n$ 。

例如 $\text{dabbb\$}$ 的循环位移排序中, 去掉哨兵后的前缀顺序为

abbb, b, bb, bbb, dabbb,

这正是 **dabbb** 的后缀字典序。

倍增思想. 已知每个位置开始、长度为 j 的子串排名后, 长度为 $2j$ 的子串可由二元组表示:

$(\text{rank}[i], \text{rank}[i+j])$.

本板子中

$\text{sa} = p$, $\text{x} = c$, y = 临时起点/旧排名, ws = 计数桶,

其中 p, c 对应 cp-algorithms 中的有序位置数组与等价类数组。主循环

$j = 0, 1, 2, 4, 8, \dots$

每轮把已知长度翻倍。第一次 $j=0$ 可视为按单字符排序, 因为初始化时已有 $\text{x}[i]=\text{s}[i]$ 。

length = 2^k

length = 2^{k-1} class = $c[i]$

length = 2^{k-1} class = $c[i+2^{k-1}]$

length = 2^k

length = 2^{k-1} class = $c[j]$

length = 2^{k-1} class = $c[j+2^{k-1}]$

基数排序. 每轮需要按

$(\text{x}[i], \text{x}[i+j])$

排序。直接比较二元组会多一个 $\log n$, 因此使用计数排序。代码先构造 y , 使其已按第二关键字有序; 再稳定地按第一关键字 $\text{x}[\text{y}[i]]$ 计数排序:

$\text{sa}[\text{--ws}[\text{x}[\text{y}[i]]]] = \text{y}[i]$.

于是每轮为 $O(n)$, 轮数为 $O(\log n)$, 总复杂度

$O(n \log n)$.

重编号. 排序完成后, 相邻后缀 $\text{a}=\text{sa}[i-1]$, $\text{b}=\text{sa}[i]$ 的新排名由旧二元组决定:

$(\text{old}[\text{a}], \text{old}[\text{a}+j]) = (\text{old}[\text{b}], \text{old}[\text{b}+j]) \Rightarrow \text{new}[\text{a}] = \text{new}[\text{b}].$

对应代码为

$(\text{y}[\text{a}] == \text{y}[\text{b}] \ \&\& \ \text{y}[\text{a}+j] == \text{y}[\text{b}+j]).$

若不同排名数 p 已达到 n , 所有后缀排名唯一, 构造结束。

LCP/height. 后缀数组常配合相邻最长公共前缀数组:

$\text{lcp}[i] = \text{LCP}(\text{s}[\text{sa}[i] \dots], \text{s}[\text{sa}[i-1] \dots]), \quad \text{lcp}[0] = 0.$

构造完 sa 后, $\text{x}[i]$ 即 $\text{rk}[i]$ 。后缀 i 在排序中的前一个后缀为

$j = \text{sa}[\text{x}[i]-1]$.

Kasai 思想: 若上一次公共前缀长度为 k , 起点右移一位后至少可从 $k-1$ 继续尝试, 因此代码中有

$\text{k} \ \&\& \ \text{k}--.$

LCP 构造总复杂度为 $O(n)$ 。

常用公式. 任意两个后缀在 **sa** 中排名为 $l < r$ 时:

$$\text{LCP}(\text{sa}[l], \text{sa}[r]) = \min_{i=l+1}^r \text{lcp}[i].$$

不同子串个数:

$$\frac{n(n+1)}{2} - \sum_i \text{lcp}[i].$$

最长重复子串:

$$\max_i \text{lcp}[i].$$

至少出现 k 次的最长子串:

$$\max_l \min_{i=l+1}^{l+k-1} \text{lcp}[i].$$

两个串最长公共子串: 构造

$$S = A + \# + B,$$

枚举相邻后缀, 若来自不同原串, 用 **lcp[i]** 更新答案。

SuffixArray.h

Description: 构造字符串的后缀数组。**sa[i]** 表示后缀数组中第 i 小后缀的起始位置。返回数组大小为 $n+1$, 且 $\text{sa}[0] = n$ 。**lcp** 数组保存后缀数组中相邻后缀的最长公共前缀: $\text{lvp}[i] = \text{lcp}(\text{sa}[i], \text{sa}[i-1])$, $\text{lcp}[0] = 0$ 。输入字符串不能包含空字符。

Time: $\mathcal{O}(n \log n)$ 635552, 22 lines

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i, 0, n) ws[x[i]]++;
            rep(i, 1, lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i, 1, n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                 s[i + k] == s[j + k]; k++);
    }
};
```

Various (6)

6.1 Misc. algorithms

LIS.h

Description: 返回最长上升 (非降) 子序列的下标。

Time: $\mathcal{O}(N \log N)$ 277a90, 17 lines

```
template<class I> vi lis(const vector<I> &S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    using p = pair<I, int>;
    vector<p> res;
    rep(i, 0, sz(S)) {
        // 把lower_bound中的 0 -> i 以求最长非降子序列
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
```

```
*it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it - 1)->second;
}
int L = sz(res), cur = res.back().second;
vi ans(L);
while (L--) ans[L] = cur, cur = prev[cur];
return ans;
}
```